

6. Herencia y escalabilidad de modelos y vistas

SISTEMAS DE GESTIÓN EMPRESARIAL

2º DAM

A solid orange horizontal bar at the bottom of the slide.

Bibliografía

[Odoo 11 Development Essentials - Third Edition](#)

ISBN-13: 978-1788477796

[Odoo 11 Development Cookbook](#)

ISBN-13: 978-1788471817

[Working with Odoo 11 - Third Edition: Configure, manage, and customize your Odoo system](#)

ISBN-13: 978-1788476959

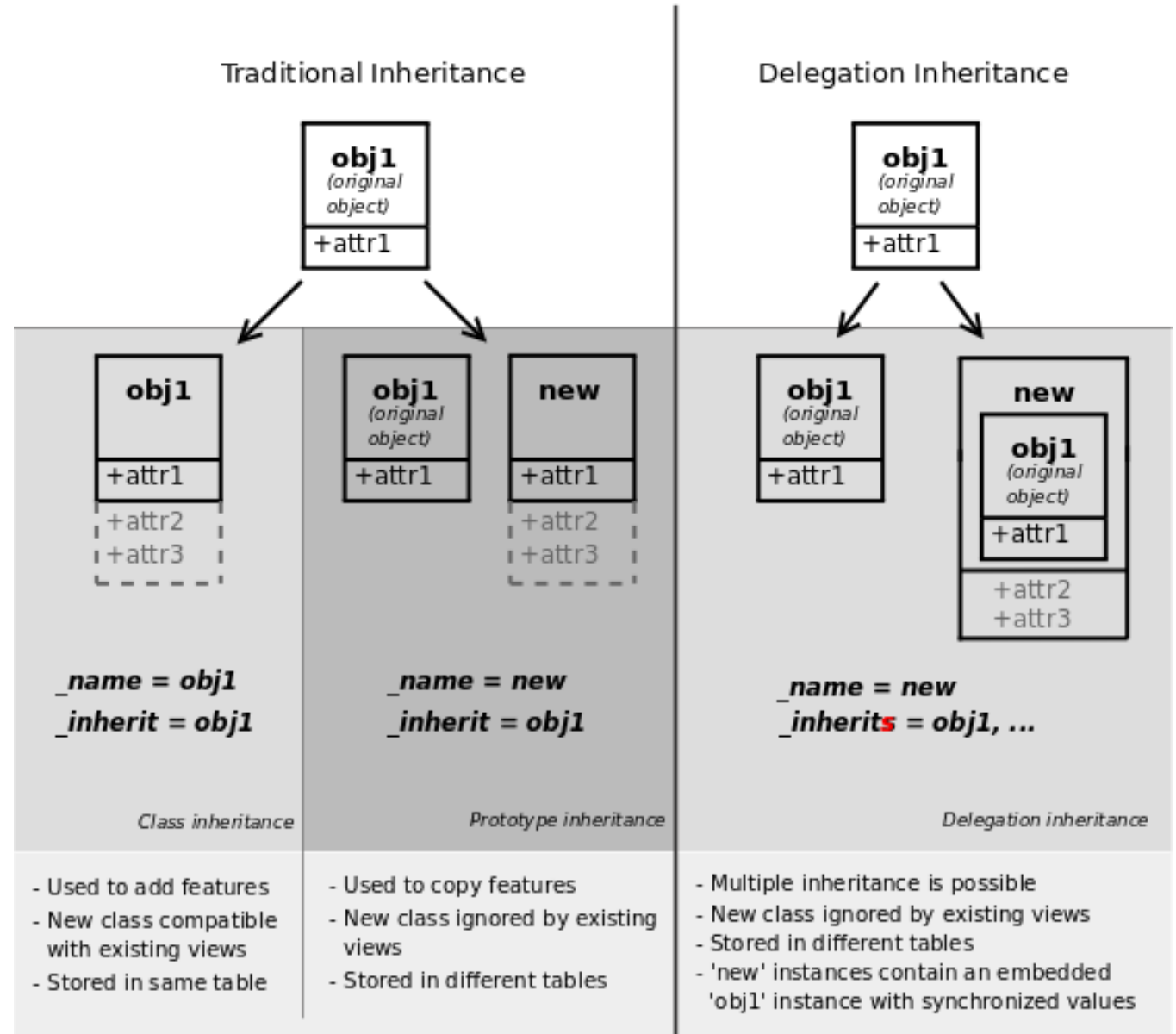
Herencia

- No se debe confundir con la herencia de clases de Python
- Hay que entenderlo como la posibilidad de extender la funcionalidad de un modelo agregando nuevos campos y actualizando las vistas con estos nuevos campos
 - Herencia de modelos (objetos)
 - ✓ Simple:
 - Por extensión
 - Por prototipo
 - ✓ Múltiple
 - Por delegación
 - Herencia de vistas

Herencia

- Odoo proporciona dos mecanismos de herencia para extender un modelo existente de forma modular.
- El primer mecanismo de herencia permite que un módulo modifique el comportamiento de un modelo definido en otro módulo:
 - ✓ Agregar campos a un modelo
 - ✓ Anular la definición de campos en un modelo
 - ✓ Agregar restricciones a un modelo
 - ✓ Agregar métodos a un modelo
 - ✓ Anular los métodos existentes en un modelo.
- El segundo mecanismo de herencia (delegación) permite vincular cada registro de un modelo a un registro en un modelo principal y proporciona acceso transparente a los campos del registro principal.

Herencia



Herencia tradicional.

Extensión

- Cuando un se define con el atributo `_inherit`, agrega modificaciones al modelo heredado en lugar de reemplazarlo.
- Esto significa que los campos definidos en la clase heredada se agregan o cambian respecto al modelo padre.
- En la capa de base de datos se agregarán campos en la misma tabla de base de datos.
- Si el campo ya existe en la superclase, se modifican con la definición en la clase hija, el resto de campos se mantienen como estaban en la clase padre.
- Los métodos definidos en la clase heredada reemplazan el método en la clase padre, que de todas maneras se podrán invocar utilizando la palabra reservada `super`.

Herencia tradicional. Extensión

```
#####  
#### libro.py en módulo Biblioteca      #####  
#####
```

```
from odoo import models, fields, api  
  
class libro(models.Model):  
    _name="biblioteca.libro" #Nombre del modelo  
  
    title = fields.Char("Título",size=128)  
    image = fields.Binary("Imagen")  
    isbn = fields.Char("ISBN",size=16)
```

```
#####  
#### libro.py en otro módulo Odoo      #####  
#####
```

```
class libro(models.Model):  
    #Nombre del modelo heredado  
    _inherit="biblioteca.libro"  
  
    # Nuevo campo agregado  
    npage = fields.Integer("Nº de páginas")  
  
    # Campo modificado  
    title = fields.Char("Título",size=64,required=True)
```

Herencia tradicional.

Herencia por prototipo

- También es posible copiar el modelo principal en un modelo completamente nuevo.
- Esto se hace simplemente agregando un modelo que tenga un atributo `_name` con un identificador diferente.
- El nuevo modelo tiene su propia tabla en la base de datos con sus propios datos totalmente independientes del modelo padre.
- Hay que tener en cuenta que las modificaciones del modelo padre afectarán al modelo hijo, modificando la estructura en la base de datos del nuevo modelo.
- En la documentación oficial, esto se llama herencia por prototipo

Herencia tradicional.

Herencia por prototipo

```
#####  
#### libro.py en módulo Biblioteca #####  
#####
```

```
from odoo import models, fields, api
```

```
class libro(models.Model):  
    _name="biblioteca.libro" #Nombre del modelo  
  
    title = fields.Char("Título",size=128)  
    image = fields.Binary("Imagen")  
    isbn = fields.Char("ISBN",size=16)
```

```
#####  
#### manual.py en otro módulo Odoo #####  
#####
```

```
class manual(models.Model):
```

```
    #Nombre del modelo padre  
    _inherit="biblioteca.libro"  
    #Nombre del modelo hijo  
    _name="biblio2.manual"  
  
    # Nuevo campo agregado  
    npage = fields.Integer("Nº de páginas")
```

Herencia por delegación.

- La herencia por delegación permite crear un nuevo modelo para incluir características de un modelo principal.
- También es compatible con la herencia polimórfica, donde heredamos de dos o más modelos diferentes.
- Cuando se crea un nuevo registro para el nuevo modelo, también se crea un nuevo registro en el modelo original y se vincula, utilizando un campo Many2one.
- Las vistas del nuevo modelo ven todos los campos: los heredados y los nuevos.
- Utiliza `_inherits`

Herencia por delegación.

```
class LibraryMember(models.Model):
    _name = 'library.member'
    _inherits = {'res.partner': 'partner_id'}

    partner_id = fields.Many2one('res.partner', ondelete='cascade')
    #Campos nuevos
    date_start = fields.Date('Member Since')
    date_end = fields.Date('Termination Date')
    member_number = fields.Char()
```

- Al nivel de la base de datos cuando se crea un nuevo miembro:
 - Se crea un nuevo registro en la tabla res_partner
 - Se crea un nuevo registro en la tabla library_member
 - El campo partner_id de la tabla library_member se establece con el id del registro res_partner que se crea para él

Herencia de vistas

- Cuando se crean objetos heredados es necesario modificar la vista para esos objetos.
- Se aplica a los casos de herencia por extensión
- Se utiliza el campo *inherit_id* para referenciar el id de la vista que se hereda.
- Los valores del atributo *position* definen cómo se modifica la vista original.
 - **inside (default)**: anexa una etiqueta dentro
 - **after**: se agrega después de la etiqueta
 - **before**: se agrega antes de la etiqueta
 - **replace**: se reemplaza el contenido de la etiqueta

Herencia de vistas.

Agregar un campo

```
<record model="ir.ui.view" id="view_partner_form">
  <field name="name">res.partner.form.inherit</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="type">form</field>
  <field name="arch" type="xml">
    <notebook position="inside">
      <page string="Relations">
        <field name="relation_ids" colspan="4" nolabel="1" />
      </page>
    </notebook>
  </field>
</record>
```

Vista de la que hereda

Indica que se agregará dentro de notebook

Página agregada

Herencia de vistas.

Reemplazar un campo

```
<record model="ir.ui.view" id="view_partner_form1">
  <field name="name">res.partner.form.inherit1</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="type">form</field>
  <field name="arch" type="xml">
    <page string="Extra Info" position="replace">
      <field name="relation_ids" colspan="4" nolabel="1" />
    </page>
  </field>
</record>
```

Vista de la
que hereda

Indica que se
reemplazará
<page>

Campo agregado
en sustitución de
<page>

Herencia de vistas.

Eliminar un campo

```
<record model="ir.ui.view" id="view_partner_form2">
  <field name="name">res.partner.form.inherit2</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="type">form</field>
  <field name="arch" type="xml">
    <field name="lang" position="replace" />
  </field>
</record>
```

Vista de la
que hereda

Indica que se reemplazará por
nada → Eliminar campo de la
vista

Herencia de vistas.

Agregar campos antes

```
<record model="ir.ui.view" id="view_partner_form3">
  <field name="name">res.partner.form.inherit3</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="type">form</field>
  <field name="arch" type="xml">
    <field name="lang" position="before">
      <field name="relation_ids" />
    </field>
  </field>
</record>
```

Vista de la
que hereda

El nuevo campo se
colocará antes del campo
"lang"

Herencia de vistas.

Agregar campos después

```
<record model="ir.ui.view" id="view_partner_form3">
  <field name="name">res.partner.form.inherit3</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="type">form</field>
  <field name="arch" type="xml">
    <field name="lang" position="after">
      <field name="relation_ids" />
    </field>
  </field>
</record>
```

Vista de la
que hereda

El nuevo campo se
colocará después del
campo "lang"

Herencia de vistas. Múltiples cambios

```
<record model="ir.ui.view" id="view_partner_form3">
  <field name="name">res.partner.form.inherit3</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="type">form</field>
  <field name="arch" type="xml">
    <field name="lang" position="replace"/>
      <field name="website" position="after">
        <field name="lang"/>
      </field>
    </field>
  </field>
</record>
```

Vista de la
que
hereda

Campo que se
elimina

El campo se vuelve a
agregar en otra
posición

Herencia de vistas.

XPath

En ocasiones, no es suficiente con el nombre del campo, por ejemplo cuando haya múltiples visualizaciones del mismo campo. Usamos **xpath** para describir dónde se ubicará el cambio

Herencia de vistas. XPath

```
<record model="ir.ui.view" id="view_partner_form6">
  <field name="name">res.partner.form.inherit6</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="arch" type="xml">
    <data>
      <xpath expr="//field[@name='address']/form/field[@name='email']" position="after">
        <field name="age" />
      </xpath>
      <xpath expr="//field[@name='address']/tree/field[@name='email']" position="after">
        <field name="age" />
      </xpath>
    </data>
  </field>
</record>
```

Añadimos "age" después de email en las vistas Tree y Form de la lista de direcciones